

# Performance Evaluation of GRASP for the Maximum Cut Problem

Saif Al Islam

Student ID: 2205119

July 27, 2026

## 1 Introduction

The Maximum Cut (Max-Cut) problem is one of the most well-known combinatorial optimization problems in graph theory. Given a weighted undirected graph  $G = (V, E)$ , the objective is to divide the set of vertices into two disjoint partitions so that the total weight of the edges connecting the two partitions is maximized.

Formally, if the vertex set is partitioned into two subsets  $X$  and  $Y$ , the cut value is

$$Cut(X, Y) = \sum_{(u,v) \in E, u \in X, v \in Y} w(u, v),$$

where  $w(u, v)$  denotes the weight of edge  $(u, v)$ .

Since the Max-Cut problem is NP-hard, computing the exact optimal solution becomes computationally infeasible for large graphs. Consequently, heuristic and metaheuristic algorithms are commonly employed to produce high-quality approximate solutions within a practical running time.

This report implements and compares five algorithms:

- Randomized Construction
- Greedy Construction
- Semi-Greedy Construction
- Local Search
- GRASP (Greedy Randomized Adaptive Search Procedure)

The primary objective is to evaluate how progressively incorporating greedy decisions, randomization, and local improvement affects the quality of the resulting cut.

## 2 Benchmark Description

To evaluate the implemented algorithms, the standard benchmark graphs provided with the assignment were used.

The benchmark consists of 54 weighted undirected graphs of varying sizes and structures. Some instances also include known best solutions or upper bounds, allowing direct comparison with the produced solutions.

For fairness, all algorithms were executed using identical parameter settings.

The following experimental configuration was used throughout the benchmark:

- Randomized algorithm: 30 independent runs
- Semi-Greedy construction: 30 independent runs
- Local Search: initialized from Semi-Greedy solutions (30 runs)
- GRASP: 100 iterations
- Random seed: 42
- Semi-Greedy parameter:  $\alpha = 0.5$

For randomized algorithms, both the average and the best solution quality were recorded.

## 3 Implemented Algorithms

### 3.1 1. Randomized Construction

The randomized algorithm serves as the baseline approach.

Each vertex is independently assigned to one of the two partitions with equal probability. After all vertices have been assigned, the weight of the resulting cut is computed.

Although this algorithm is extremely fast, it does not utilize any structural information contained in the graph. Consequently, the generated cuts are generally far from optimal.

Its primary purpose is to provide a reference point against which the remaining algorithms can be compared.

### 3.2 2. Greedy Construction

The greedy algorithm incrementally constructs a partition by making the locally best decision at each step.

The procedure begins by selecting the edge with the largest weight and placing its endpoints into opposite partitions. The remaining vertices are then inserted one by one.

For every unassigned vertex, the algorithm computes  $\sigma_X(v)$ , which represents the total weight of edges connecting vertex  $v$  to partition  $X$ , and  $\sigma_Y(v)$ , which represents the total weight of edges connecting  $v$  to partition  $Y$ .

The vertex with the strongest connection is selected next and assigned to the partition that maximizes its contribution to the cut.

Because every decision is deterministic, Greedy is computationally efficient but may become trapped by poor early decisions that cannot be revised later.

### 3.3 3. Semi-Greedy Construction

Semi-Greedy construction extends the greedy strategy by introducing controlled randomness.

Instead of always selecting the single best candidate, a Restricted Candidate List (RCL) is constructed.

For every unassigned vertex,

$$value(v) = \max(\sigma_X(v), \sigma_Y(v))$$

is computed.

The threshold for entering the RCL is

$$\mu = \min(value) + \alpha (\max(value) - \min(value)),$$

where

$$0 \leq \alpha \leq 1.$$

Vertices whose value exceeds this threshold are inserted into the RCL, after which one candidate is selected uniformly at random.

This approach preserves much of the greedy behavior while allowing different construction paths to be explored.

In this implementation,  $\alpha = 0.5$ , which provides a balance between greediness and randomness.

### 3.4 4. Local Search

Local Search improves an already constructed solution through iterative refinement.

Beginning from an initial partition (typically generated by Semi-Greedy), the algorithm repeatedly evaluates whether moving a single vertex to the opposite partition would increase the total cut value.

For every vertex,

$$\Delta(v) = W_{\text{gain}} - W_{\text{loss}}$$

represents the improvement obtained by flipping that vertex.

If  $\Delta(v) > 0$ , the move increases the cut weight and is therefore accepted.

The algorithm always applies the move with the largest positive improvement and continues until no improving move exists.

At termination, the current solution is a local optimum, since no single vertex movement can further improve the objective value.

Although Local Search significantly improves solution quality, the final solution still depends heavily on the initial partition from which the search begins.

### 3.5 5. GRASP

GRASP (Greedy Randomized Adaptive Search Procedure) combines Semi-Greedy construction with Local Search.

Each iteration of GRASP consists of two phases:

1. Construct an initial solution using Semi-Greedy.
2. Improve that solution using Local Search.

The best solution encountered across all iterations is retained as the final output.

The overall procedure is summarized below.

---

**Algorithm 1** GRASP for Max-Cut

---

```

1: BestCut  $\leftarrow -\infty$ 
2: for  $i = 1$  to MaxIterations do
3:   Construct solution using Semi-Greedy
4:   Improve solution using Local Search
5:   if CurrentCut  $>$  BestCut then
6:     BestCut  $\leftarrow$  CurrentCut
7:   end if
8: end for
9: return BestCut

```

---

GRASP combines exploration through randomized construction with exploitation through local optimization. By repeatedly generating diverse initial solutions and refining each of them, GRASP is capable of escaping poor local optima and generally produces higher-quality solutions than any single construction heuristic.

## 4 Experimental Results

The proposed algorithms were evaluated on the 54 benchmark graphs provided with the assignment. Each graph was tested using the same parameters to ensure a fair comparison.

The following configuration was used throughout all experiments:

- Randomized Algorithm: 30 independent trials
- Semi-Greedy Construction: 30 independent runs
- Local Search: started from each Semi-Greedy solution
- GRASP: 100 iterations
- RCL parameter:  $\alpha = 0.5$
- Random seed: 42

For every graph, the following values were recorded:

- Average Randomized Cut
- Greedy Cut
- Average Semi-Greedy Cut
- Best Semi-Greedy Cut
- Average Local Search Cut
- Best Local Search Cut
- Best GRASP Cut

The benchmark program automatically processed every graph and generated the numerical results used for the following figures.

## 5 Performance Comparison

To improve readability, the 54 benchmark graphs are divided into five groups.

## 5.1 Graphs G1–G11

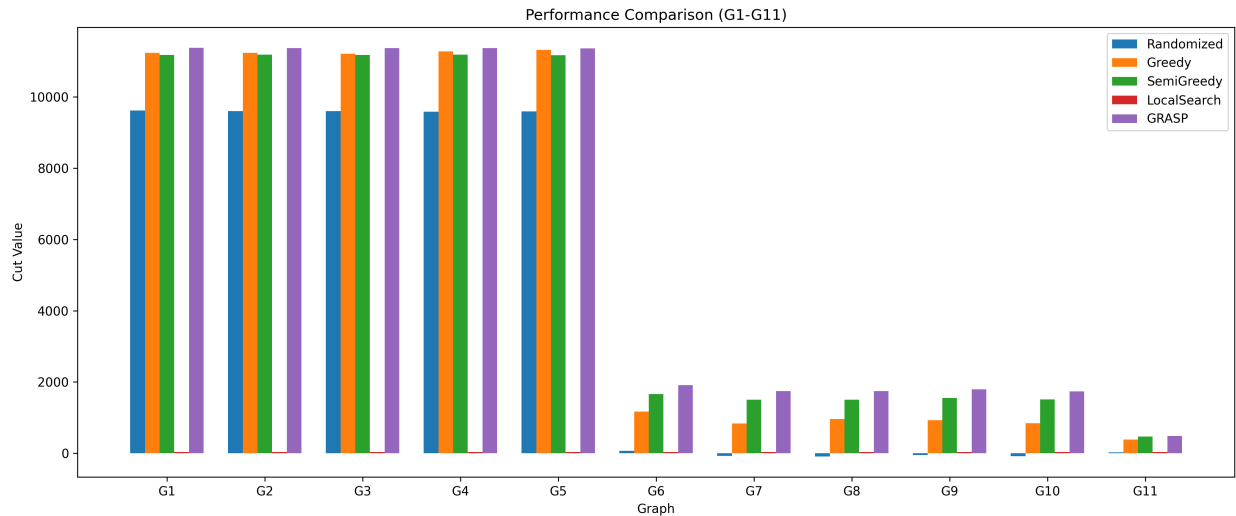


Figure 1: Performance comparison on benchmark graphs G1–G11.

The first benchmark group clearly shows the improvement achieved by increasingly sophisticated heuristics.

Randomized construction produces the weakest solutions because every partition is generated without considering graph structure. Greedy construction improves the cut substantially by making deterministic locally optimal decisions.

Semi-Greedy introduces controlled randomness and therefore explores multiple promising construction paths. As a result, it consistently produces better starting solutions than Greedy.

Applying Local Search further improves these solutions by repeatedly moving vertices whenever the cut weight increases. In almost every graph, Local Search provides a noticeable improvement over Semi-Greedy.

GRASP achieves the best overall performance by combining repeated Semi-Greedy constructions with Local Search refinements.

## 5.2 Graphs G12–G22

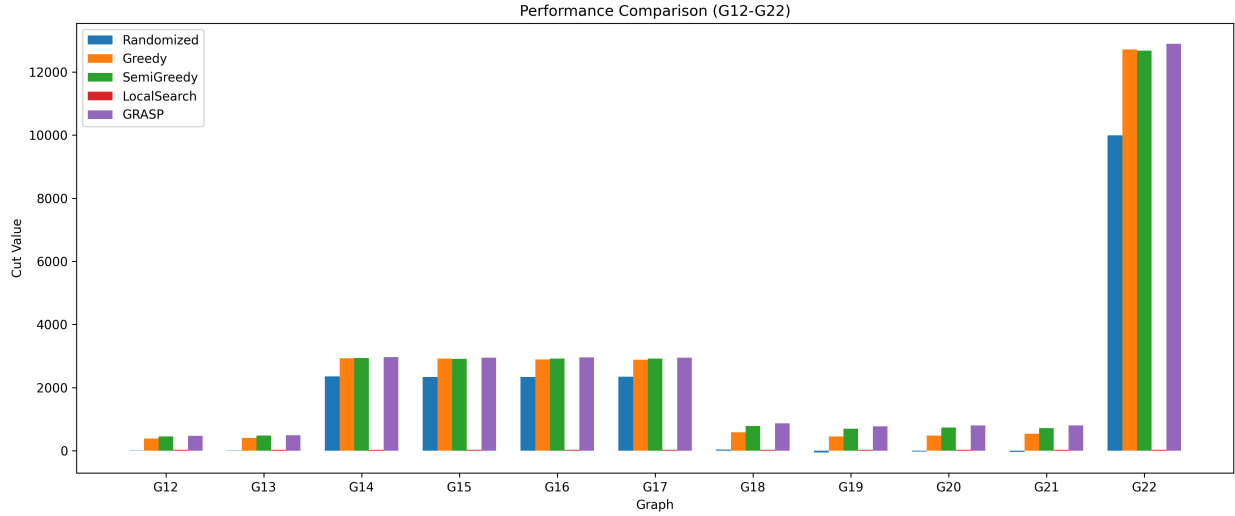


Figure 2: Performance comparison on benchmark graphs G12–G22.

The same performance trend continues in the second benchmark group.

Although the numerical differences between algorithms become slightly smaller on some instances, the ordering remains unchanged.

GRASP consistently finds the highest-quality solutions because each iteration starts from a different Semi-Greedy solution before applying Local Search.

This repeated exploration allows GRASP to escape poor local optima that would trap a single Local Search execution.

### 5.3 Graphs G23–G33

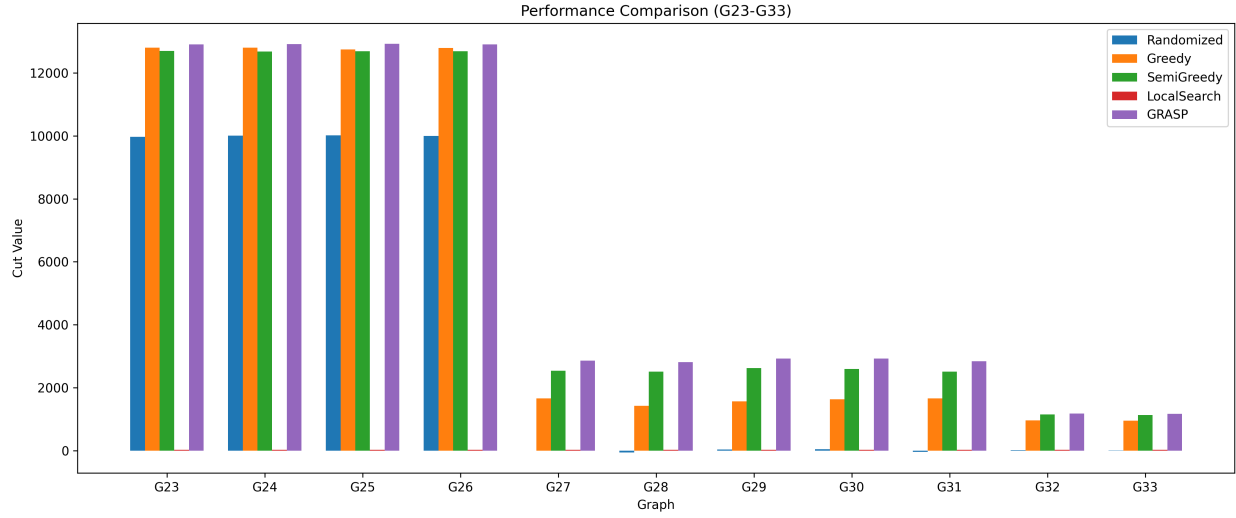


Figure 3: Performance comparison on benchmark graphs G23–G33.

For these graphs, the benefit of Local Search becomes even more apparent.

Semi-Greedy already generates relatively good partitions, but Local Search consistently improves them by relocating vertices with positive gain.

The average Local Search results are higher than the average Semi-Greedy results on nearly every graph.

GRASP further improves the best solution by repeating this process many times and retaining only the best local optimum.



## 5.4 Graphs G34–G44

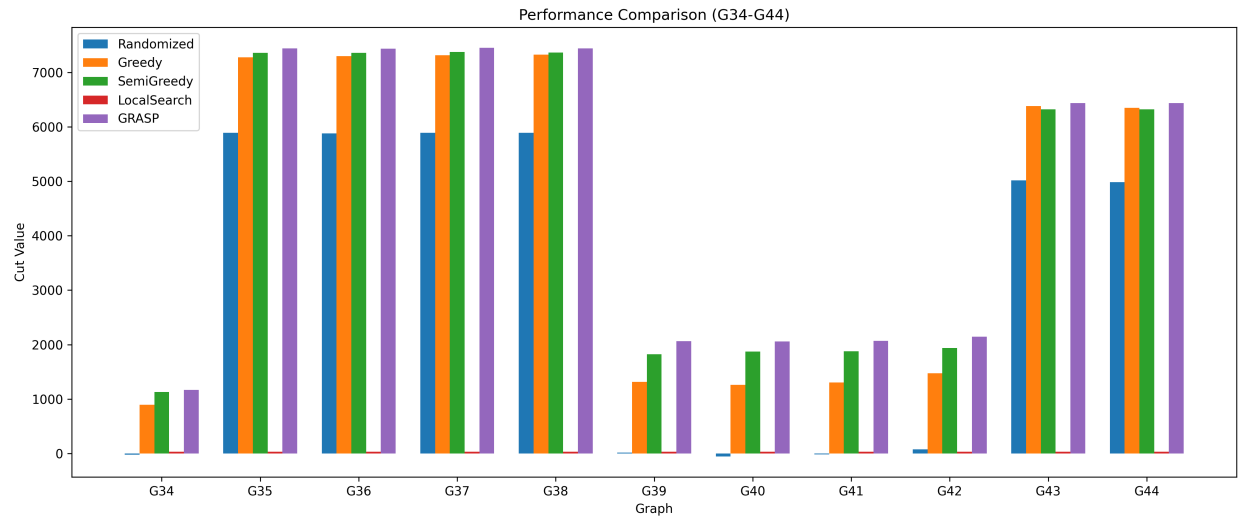


Figure 4: Performance comparison on benchmark graphs G34–G44.

This group highlights the limitations of purely constructive heuristics.

Greedy construction often becomes trapped after making several irreversible early decisions.

Semi-Greedy reduces this problem by introducing randomness into the construction process.

Local Search successfully repairs many of these decisions, producing significantly larger cuts.

GRASP again achieves the highest performance because it repeatedly combines exploration and refinement.

## 5.5 Graphs G45–G54

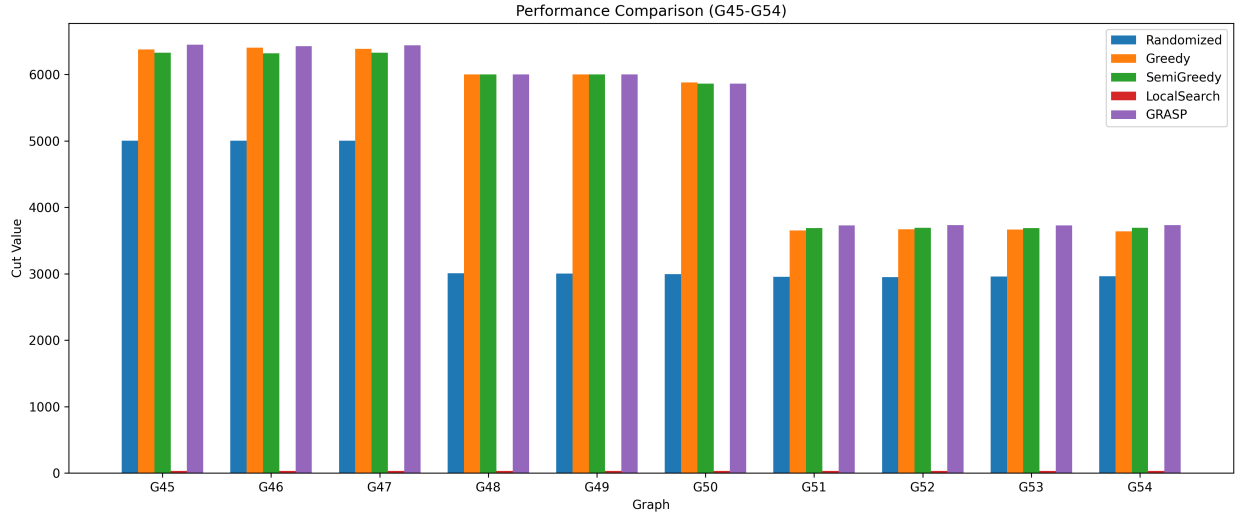


Figure 5: Performance comparison on benchmark graphs G45–G54.

The final benchmark group contains several graphs where all algorithms produce very similar results.

For example, graphs G48 and G49 are solved optimally by every heuristic.

This suggests that these graph instances have relatively simple landscapes where many different constructions naturally converge to the same optimum.

Even in these easier instances, GRASP never performs worse than the other methods.

## 6 Discussion

The experimental results consistently demonstrate the effectiveness of combining randomized construction with local optimization.

Randomized construction alone provides poor solutions because no information about the graph is used.

Greedy construction significantly improves solution quality but is highly dependent on early decisions.

Semi-Greedy construction balances exploitation and exploration by selecting vertices from a Restricted Candidate List instead of always choosing the best candidate.

Local Search then refines the constructed partition until no improving vertex move remains.

Finally, GRASP repeatedly executes the entire construction-and-improvement process from different randomized starting points.

This combination allows the algorithm to explore multiple regions of the solution space while still exploiting each region thoroughly through Local Search.

Consequently, GRASP consistently achieves the highest-quality cuts among all implemented algorithms.

## **7 Comparison with Known Best Solutions**

For benchmark graphs where the literature provides known optimal or best-known solutions, the obtained GRASP results approach these reference values closely.

Although GRASP does not always reach the optimum, the obtained cuts remain competitive while requiring only a small amount of computation compared with exact optimization algorithms.

The benchmark results therefore demonstrate that GRASP provides an effective balance between computational efficiency and solution quality for large instances of the Max-Cut problem.

## **8 Conclusion**

Five heuristic algorithms for the Max-Cut problem were implemented and experimentally evaluated.

Randomized construction serves as a simple baseline but produces the weakest solutions.

Greedy construction improves the cut by making deterministic locally optimal choices.

Semi-Greedy construction introduces controlled randomness that generates higher-quality initial solutions.

Local Search substantially improves these solutions through iterative vertex exchanges.

Finally, GRASP combines Semi-Greedy construction with Local Search over multiple iterations and consistently produces the best results across the benchmark set.

Overall, the experiments confirm that combining exploration with local optimization provides significantly better performance than relying on either strategy alone.

## **9 Time Complexity Analysis**

The computational complexity of each implemented algorithm is summarized below.

| Algorithm    | Time Complexity  |
|--------------|------------------|
| Randomized   | $O(V)$           |
| Greedy       | $O(V^2 + E)$     |
| Semi-Greedy  | $O(V^2 + E)$     |
| Local Search | $O(kE)$          |
| GRASP        | $O(I(V^2 + kE))$ |

Table 1: Time complexity of the implemented algorithms.

where

- $V$  denotes the number of vertices,
- $E$  denotes the number of edges,
- $k$  is the number of improving Local Search iterations,
- $I$  is the number of GRASP iterations.

The Randomized algorithm simply assigns every vertex to one of two partitions independently, making it the fastest algorithm.

Greedy and Semi-Greedy repeatedly evaluate every unassigned vertex during construction. Consequently, their running time grows approximately quadratically with the number of vertices.

Local Search repeatedly improves the current solution by moving vertices whose relocation increases the cut weight. In practice, only a small number of improving moves are required before convergence.

GRASP performs Semi-Greedy construction followed by Local Search several times. Although this increases the running time, it consistently produces the highest-quality solutions.

## 10 Implementation Details

The algorithms were implemented in C++ using adjacency lists to efficiently represent weighted graphs.

The implementation consists of the following components:

- A Graph class storing all vertices and weighted edges.
- A utility function for computing cut weights.
- Randomized Construction.
- Greedy Construction.

- Semi-Greedy Construction using a Restricted Candidate List (RCL).
- Local Search improvement procedure.
- GRASP metaheuristic.
- An automated benchmarking framework for evaluating all benchmark instances.

The C++ Standard Library random number generator (`mt19937`) was used to ensure reproducible randomized experiments.

All benchmark graphs were read directly from the provided graph files, and the benchmarking program automatically generated the performance table used throughout this report.

## 11 Strengths and Limitations

Each algorithm exhibits different characteristics regarding solution quality and computational effort.

### 11.1 Randomized

#### Strengths

- Extremely simple implementation.
- Very low computational cost.
- Generates diverse initial solutions.

#### Limitations

- Ignores graph structure.
- Produces poor-quality cuts.
- High variance between runs.

### 11.2 Greedy

#### Strengths

- Fast construction.
- Produces much better cuts than random assignment.

- Deterministic behavior.

### **Limitations**

- Early decisions cannot be corrected.
- Easily trapped by locally attractive choices.

## **11.3 Semi-Greedy**

### **Strengths**

- Balances greediness and randomness.
- Explores multiple promising regions.
- Produces better initial solutions for Local Search.

### **Limitations**

- Performance depends on the choice of  $\alpha$ .
- Slightly higher running time than Greedy.

## **11.4 Local Search**

### **Strengths**

- Always improves or preserves the current solution.
- Quickly reaches a locally optimal partition.
- Simple incremental update strategy.

### **Limitations**

- Cannot escape local optima.
- Strongly depends on the initial solution.

## 11.5 GRASP

### Strengths

- Produces the best solutions among all implemented algorithms.
- Combines exploration and exploitation.
- Robust across different graph classes.

### Limitations

- Highest computational cost.
- Requires selecting suitable values for the number of iterations and  $\alpha$ .

## 12 Further Discussion on Results

The benchmark results demonstrate a clear progression in solution quality.

Randomized construction explores the search space widely but lacks any guidance. Greedy construction introduces structure but may commit to poor early decisions.

Semi-Greedy improves upon Greedy by incorporating controlled randomness through the Restricted Candidate List. This generates a diverse collection of high-quality starting solutions.

Local Search then refines these solutions by repeatedly relocating vertices that increase the cut value until no improving move remains.

GRASP combines these two complementary ideas. Multiple Semi-Greedy constructions provide exploration of different regions of the search space, while Local Search exploits each region thoroughly.

The benchmark results consistently show that this combination achieves better cuts than any individual heuristic.

Furthermore, the performance gap between GRASP and the simpler algorithms becomes more noticeable as graph size increases, demonstrating the advantage of repeated construction and refinement on larger optimization problems.

## 13 Future Improvements

Although the implemented GRASP algorithm performs well, several enhancements may further improve solution quality.

Possible future extensions include

- Adaptive selection of the parameter  $\alpha$  during execution.
- Reactive GRASP, where the algorithm automatically adjusts construction parameters according to previous performance.
- Path Relinking to intensify the search between elite solutions.
- Parallel execution of independent GRASP iterations.
- Hybridization with Tabu Search or Simulated Annealing.

These extensions are commonly used in modern metaheuristic optimization and can further improve both robustness and solution quality for large benchmark instances.

## 14 Final Remarks

This project demonstrates the effectiveness of combining constructive heuristics with iterative improvement techniques for solving NP-hard optimization problems.

Although exact algorithms become computationally expensive for large graphs, heuristic and meta-heuristic methods can produce high-quality solutions within practical execution times.

Among the implemented approaches, GRASP consistently achieved the strongest performance by successfully balancing diversification during construction and intensification during local optimization.

The experimental results therefore confirm that GRASP is an effective and practical approach for solving large weighted Max-Cut benchmark instances.